



DISTRIBUTED DEEP LEARNING INFERENCE AT EDGE DEVICES

Final Year Project - Progress Evaluation - Group 44

K.M.B.G.B Gunaratne
M.V.R Perera
T.M.C.P Thennakoon

01

INTRODUCTION

Overview of the proposed project idea

02

PROBLEM STATEMENT

What is the problem we are going to address

03

MOTIVATION

How useful and relevant the research

04

OBJECTIVES

Planned outcomes of the project on completion

05

LITERATURE REVIEW

Evaluation of related literature

06

SOLUTION / METHODOLOGY

How we are going to provide a solution

07

PROGRESS

What we did yet

INTRODUCTION

- Deep Neural Networks are used to process large data sets.
- Requires lot of data and computational power.
- Using pre trained neural network is called *inference*.
- Edge computing ensures privacy, low latency and availability.
- But resource constrained IoT devices can not handle large neural networks.
- Distributed processing in IoT cluster may help to overcome this issue.





PROBLEM STATEMENT

Executing inference tasks in resource constrained edge devices is a challenging task.

Storing and execution of a large complete Deep Neural Network on a single edge device is infeasible. Even a medium scale DNN may present intolerable delays in executing inference calculations on these devices.



PROBLEM STATEMENT

Following table includes some results of the inference task performed by us, using a single Raspberry Pi device.

Software Framework	MobileNet v1 Benchmark value (ms)	MobileNet v2 Benchmark value (ms)	Time taken for Inception model Custom tested value (s)
TensorFlow Lite	271.5	379.6	24
TensorFlow	480.3	654.0	32



PROBLEM STATEMENT

- Lack of conducted research in this area to provide model partitioning and distributed execution in resource constrained IoT devices.
- **So our problem is to find a more efficient way to run large deep neural networks and improve the overall performance at distributed Deep Learning Inference on low resource IoT edge devices.**

MOTIVATION

- Performing inference tasks on edge devices ensure privacy of input data and can result in shorter latency when compared to a cloud solution.
 - Number of IoT devices is predicted to be rapidly increasing and reaching more than 75 billion by 2025
-
- Lack of enough research conducted to address above problems.
 - Model parallelism based partitioning has been used for training and therefore this seems a feasible solution for inference partitioning.



RESEARCH OBJECTIVES



+ Discover model parallelism based partitioning scheme for fully connected neural networks

+ Propose task distribution and task sharing mechanism among resource constrained edge devices

+ Implement proof of concept DNN/CNN partitioning and distributed processing application using python

+ Comparison of developed System with existing Systems

LITERATURE REVIEW

Types of solutions used so far for inference

Cloud based :

Latency, Availability, Scalability and Privacy issues.
When partially used with edge devices can reduce those issues.

- DDNN - 2017 (used model partitioning)

Model Pruning :

- DeepIoT (2017)

Causes Less
Accuracy

Using an Edge Server :

- RF sensing in the Internet of Things - 2018
- Deep Learning Based Food Recognition System - 2018 (with data preprocessing)
- Mainstream - 2018 (with transfer learning)

Causes resource management in the edge server when shared with multiple devices.
A trade-off between Latency and Accuracy is achieved.

LITERATURE REVIEW



Intelligent offloading across Edge Devices:

- Neurosurgeon - 2017 (layer-wise partitioning)
- DeepThings - 2018 (input-wise partitioning)
- MoDNN - 2017 (input-wise partitioning)
- Fully Distributed Deep Learning Inference on Resource-Constrained Edge Devices - 2019

SOLUTION



01

Model Parallelism based
network partitioning
scheme



02

Communication
optimization and data
distribution



03

Task allocating system
among edge Devices



SOLUTION

- Layers in a neural network need to be carefully partitioned to balance the amount of computation and memory usage on each resource constrained edge device
- We intend to research and propose a novel approach to partition and distributively execute inference calculation on multiple edge devices which are connected in a local network.
- Furthermore, we plan to investigate how to efficiently reduce the communication demands and reduce communication overhead among devices.

PROJECT STAGES



STAGE I

Domain Familiarization and exploring related works.

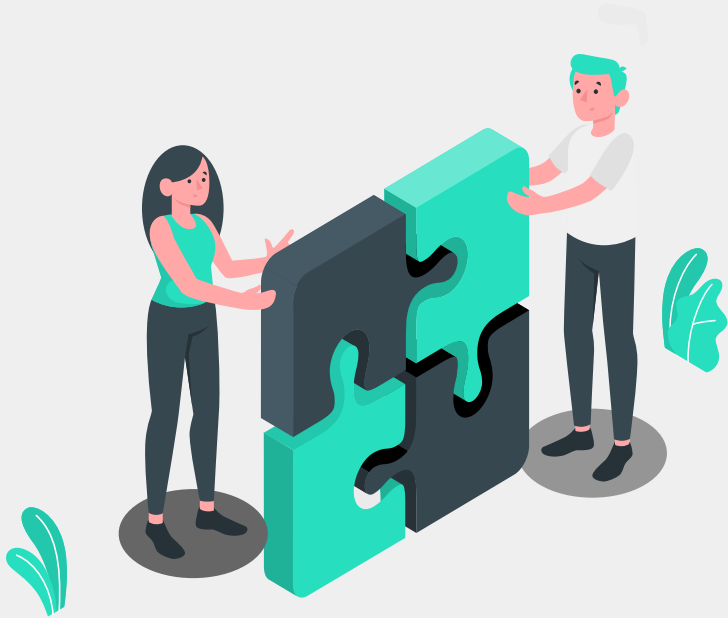
Finding possible solutions to the problem

STAGE 2

Implementation and development of the concept.

STAGE 3

Evaluation , Documentation and Paper submission.



PROGRESS

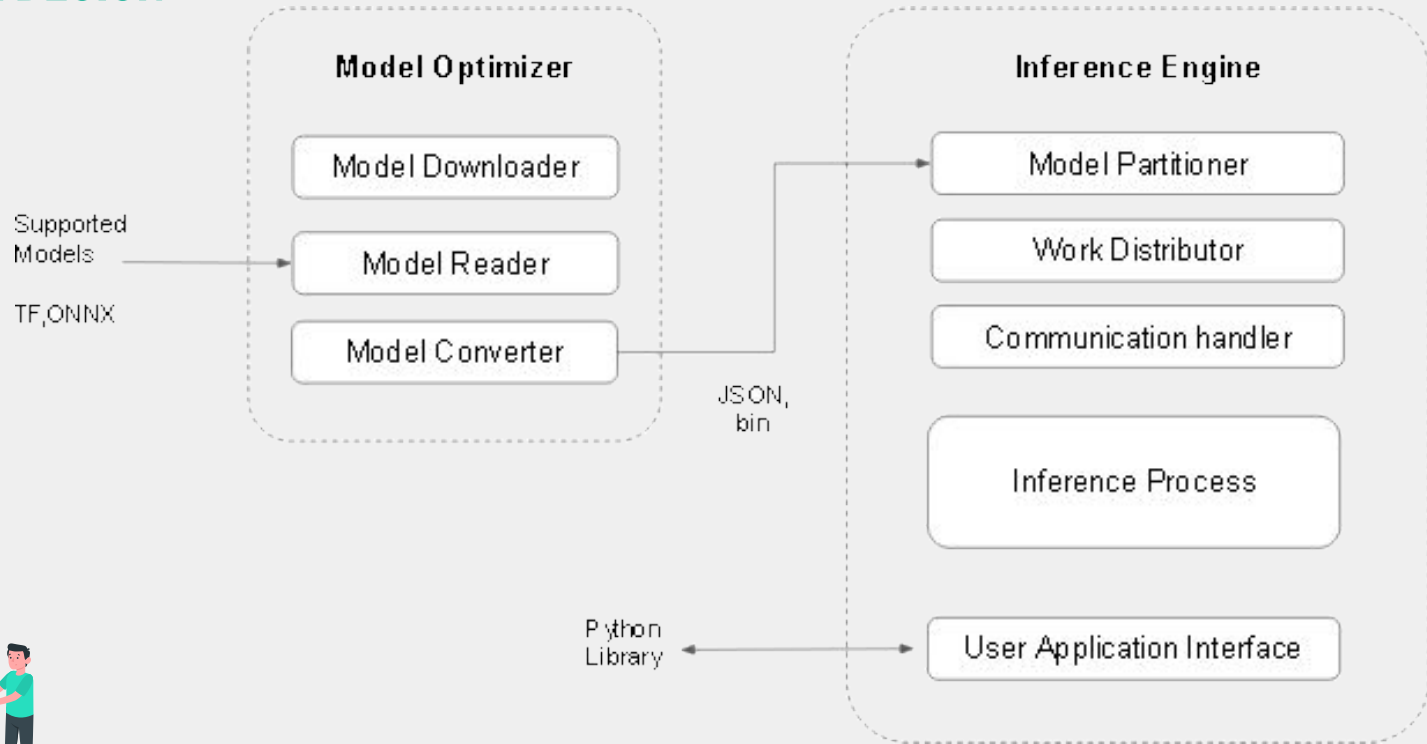
- Introduction
- System Design
- Experimental evaluation

INTRODUCTION

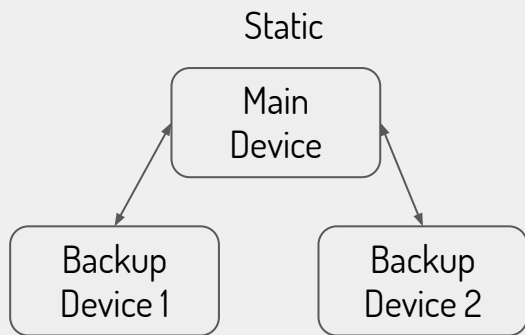
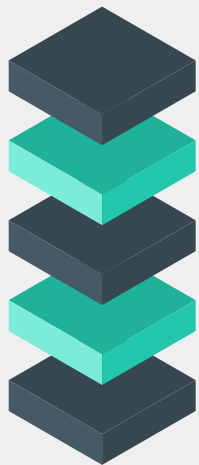
- There are many Neural Network frameworks like Tensorflow, Keras, Darknet
 - Models has two phases as Training and Inference
-
- Pre Trained models comes as files
 - Which include model architecture and weights



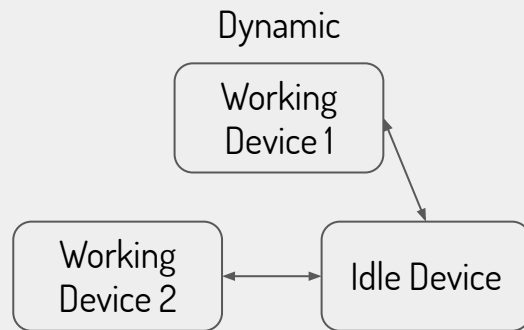
SYSTEM DESIGN



TASK DISTRIBUTION



Fixed number of backup devices
available at all time



Distribution will depend on the
number of idle devices on the
network

MODEL OPTIMIZER

Following model types were used.

1. YOLO v3
2. ONNX - Alexnet
3. Tensorflow



PROGRESS

DISTRIBUTION OF CONVOLUTION LAYERS

- Method 1
 - Only main thread
 - All the execution happens serially
- Method 2
 - Main thread with secondary thread
 - Data sending part happens parallelly

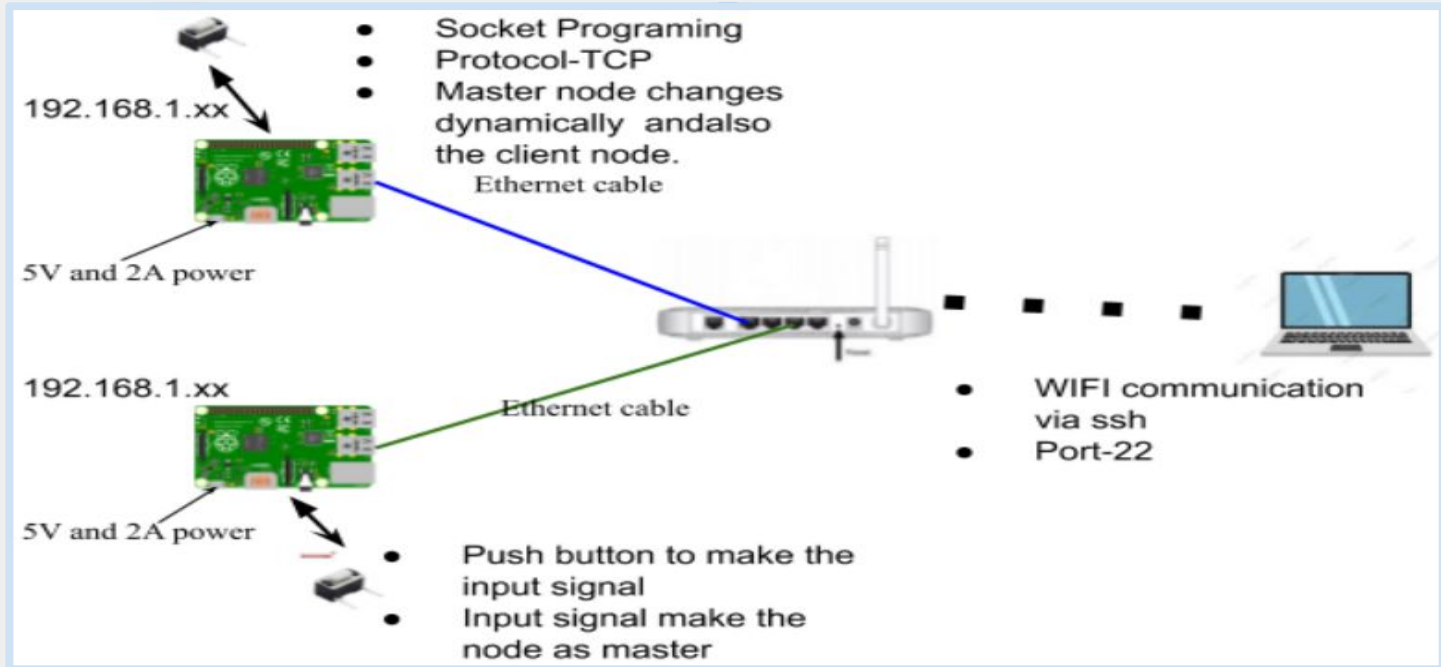
BUILDING A RASPBERRY PI CLUSTER

- Creating a Raspberry Cluster
 - 2 Raspberry Pi Devices were used
 - Data communication (Ethernet vs Wifi)
- Checking for Communication Delay

BUILDING A RASPBERRY PI CLUSTER



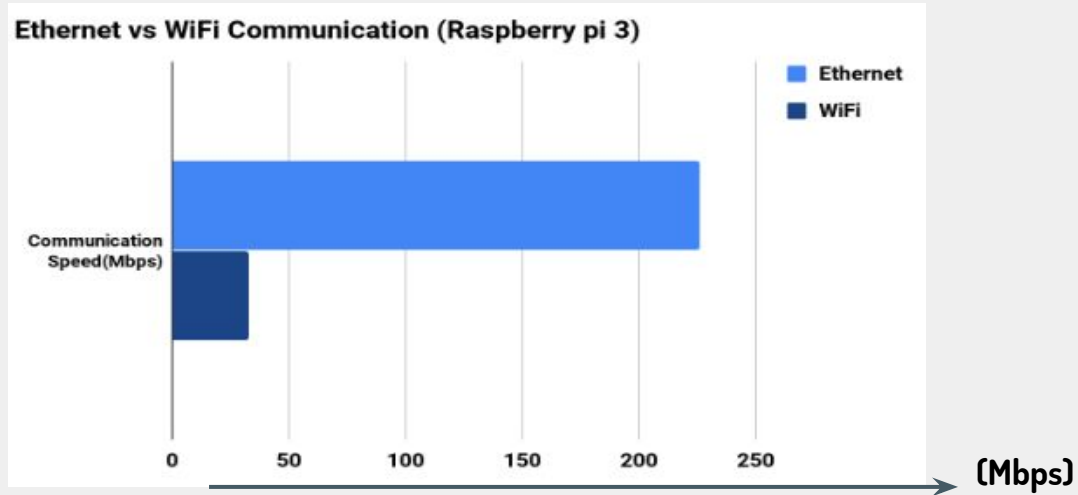
MAKING RASPBERRY PI CLUSTER



BUILDING A RASPBERRY PI CLUSTER



ETHERNET vs WIFI COMMUNICATION

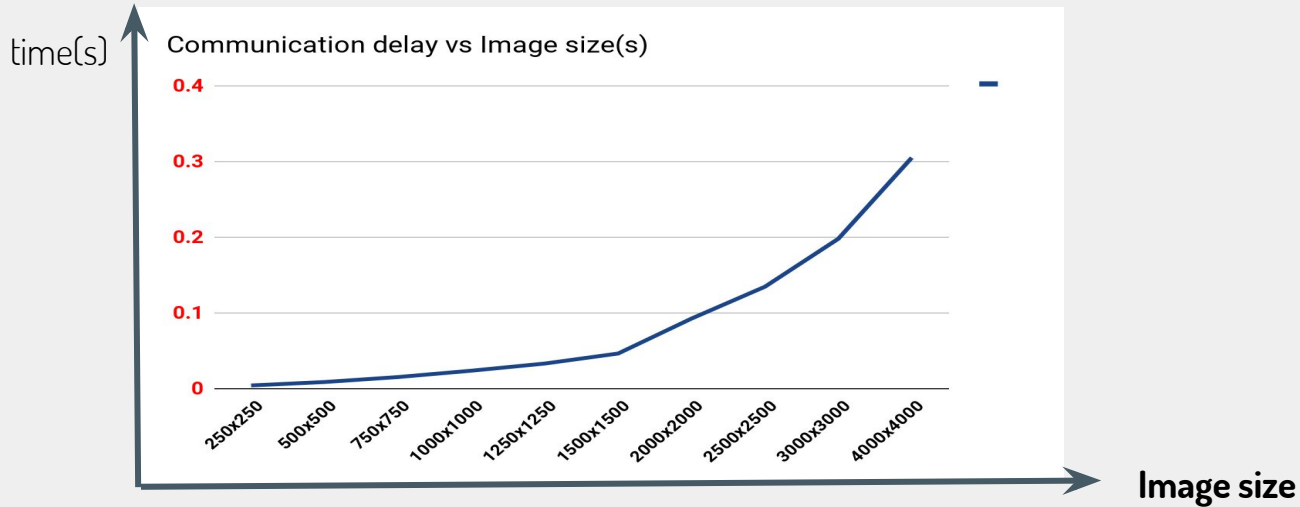


Wired communication (ethernet) is chosen for Pi cluster.

BUILDING A RASPBERRY PI CLUSTER

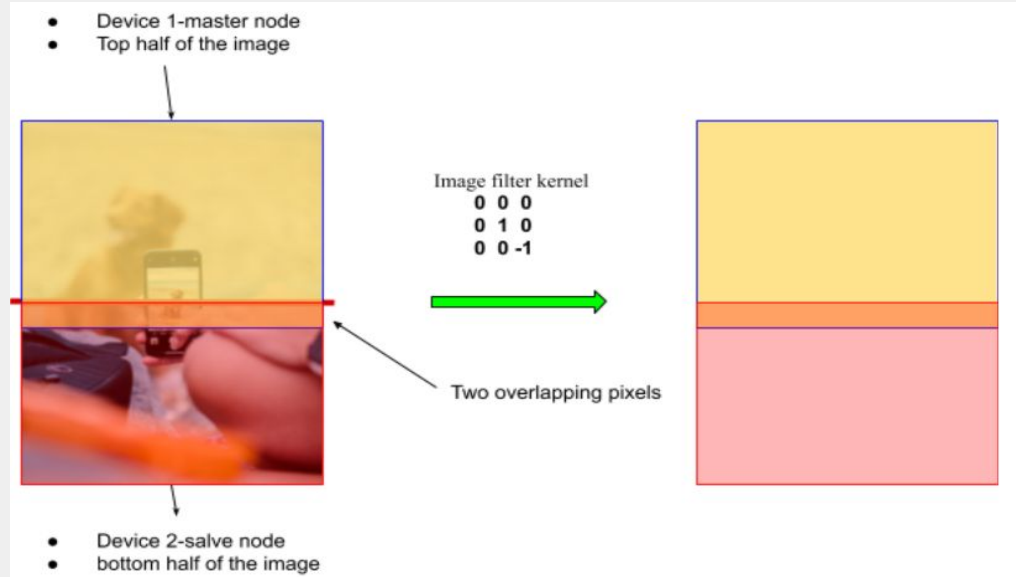


COMMUNICATION DALY



- Communication daly increases when the image size increase.

DISTRIBUTION OF CONVOLUTION LAYERS



Divide image into number of devices and send the image data as string to all slave devices.

DISTRIBUTION OF CONVOLUTION LAYERS



PYTHON LIBRARIES WE USED

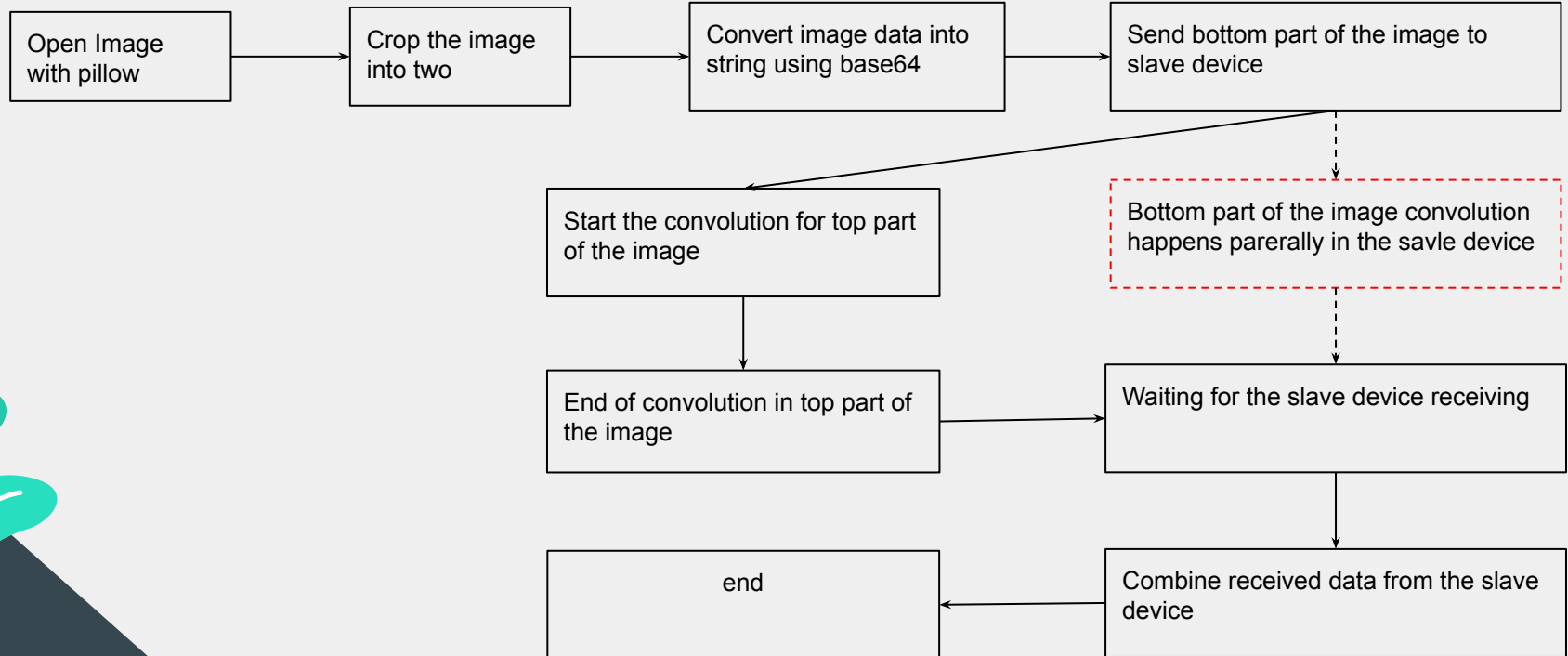
For the convolution we used following libraries

- Pillow
 - PIL Image
 - Load the image as pillow Image
 - Pillow image crop used to divide the image into two
 - Pillow image paste is used to concat the image
 - PIL ImageFilter
 - For the convolution part
- Base64
 - Convert the image data in to string by b64encode
 - Convert the image data string in image by b64decode

DISTRIBUTION OF CONVOLUTION LAYERS



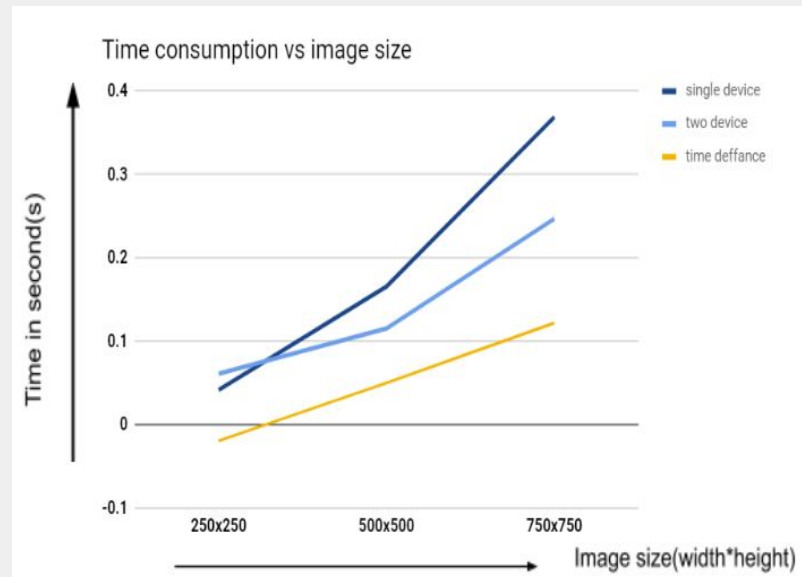
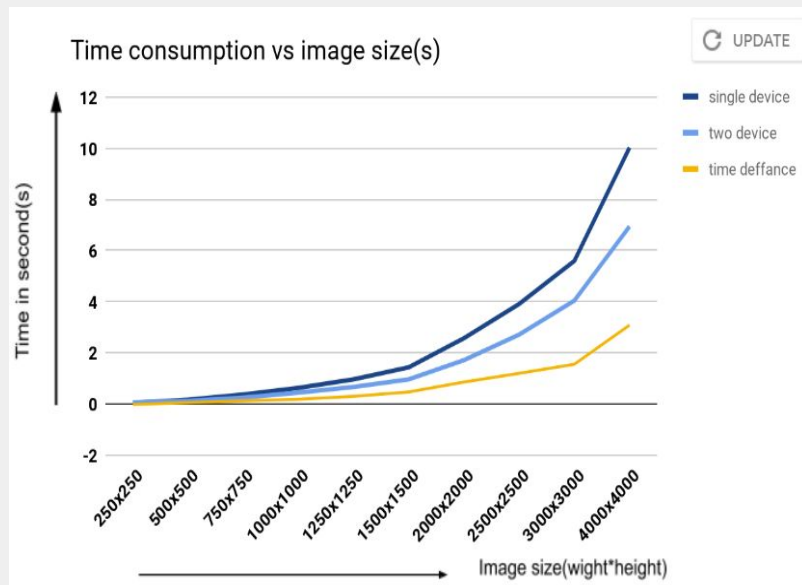
METHOD I (WORKING FLOW)



DISTRIBUTION OF CONVOLUTION LAYERS



TIME CONSUMPTION IN METHOD I

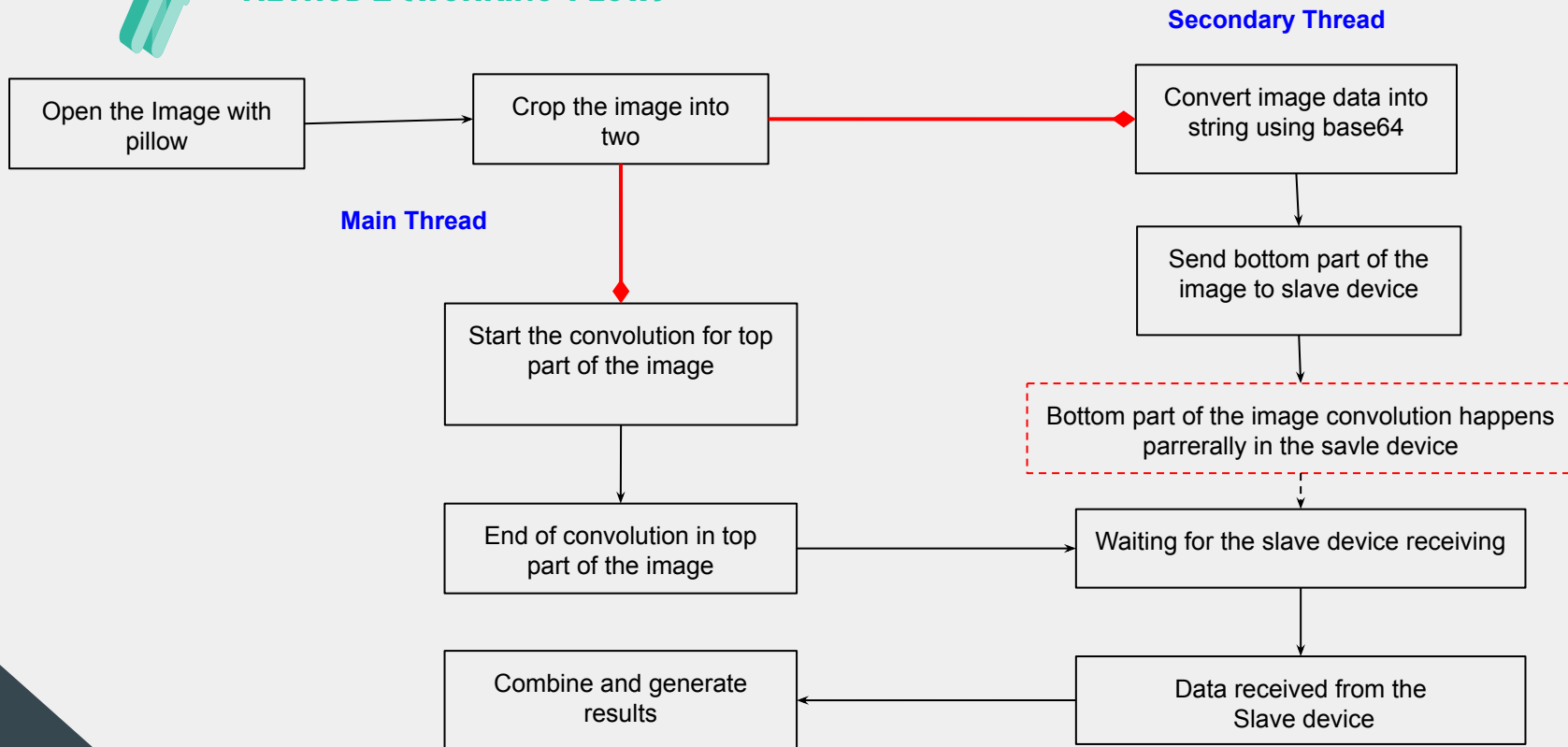


- At the point 250x250 distributed system shows less performance because communication cost get too large.

DISTRIBUTION OF CONVOLUTION LAYERS



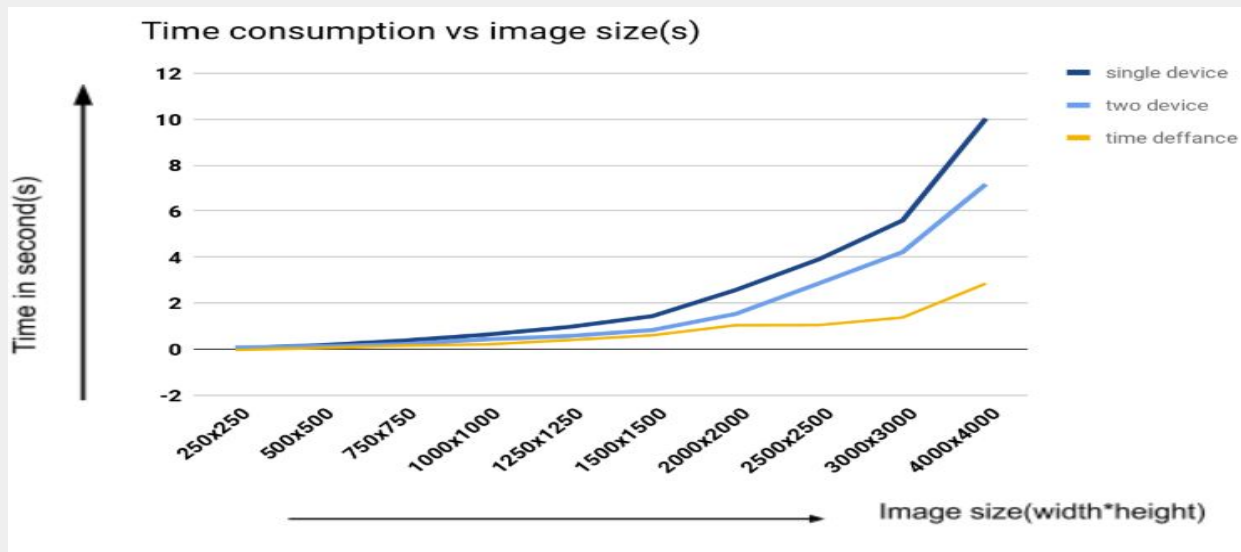
METHOD 2 (WORKING FLOW)



DISTRIBUTION OF CONVOLUTION LAYERS



TIME CONSUMPTION IN METHOD 2

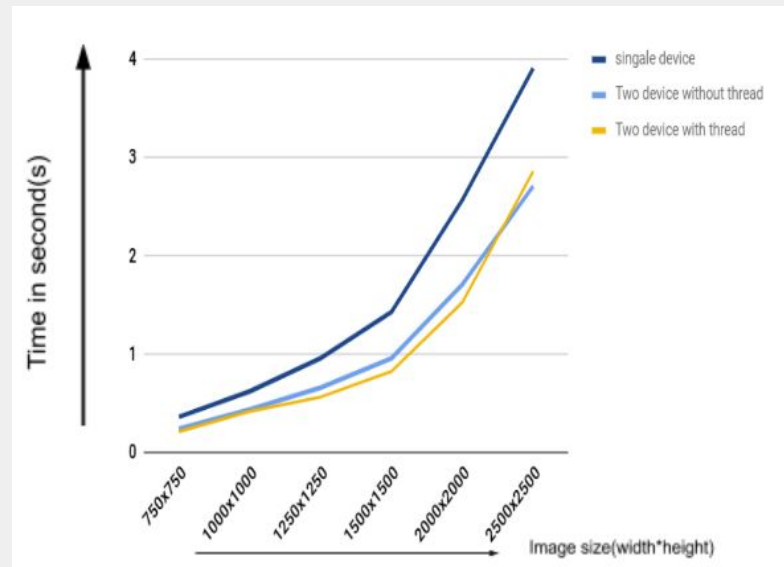
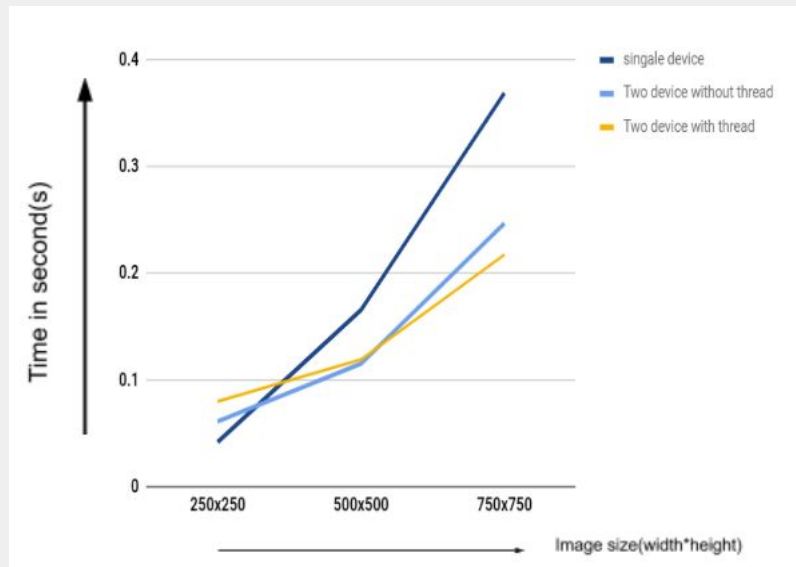


- Threaded method 2 shows the same behavior as the method 1, threaded method is also shows the performance than single device method

DISTRIBUTION OF CONVOLUTION LAYERS

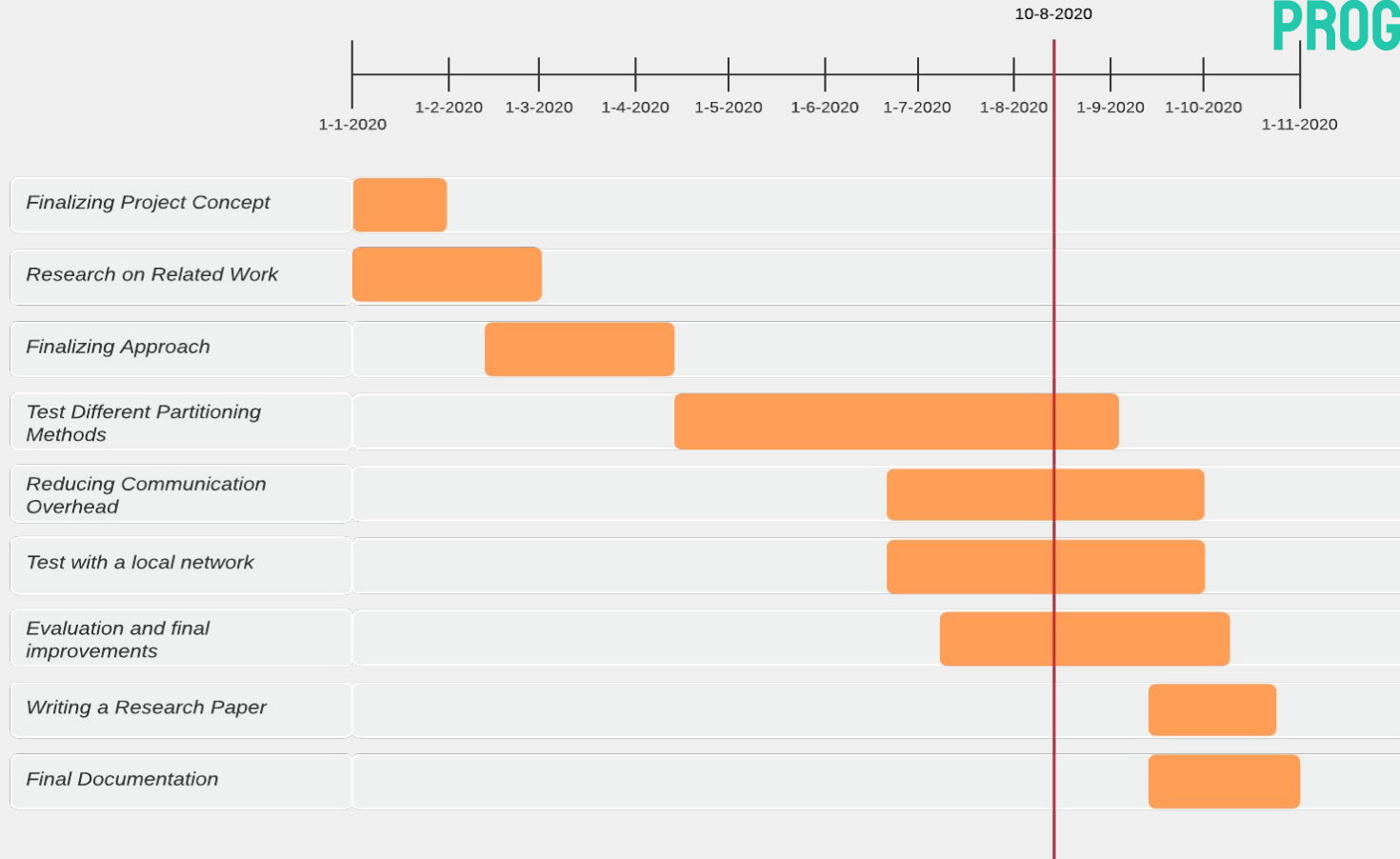


METHOD 1 VS METHOD 2 TIME CONSUMPTION



- Threaded method 2 shows the better performance when the image size is in between 500x500 - 2000x2000.
- However at point 250x250, method 2 gives worst performance

PROGRESS



REFERENCES

- J. Chen and X. Ran, "Deep Learning With Edge Computing: A Review," Proc. IEEE, vol. 107, no. 8, 2019.
- S. Liu, Y. Lin, Z. Zhou, K. Nan, H. Liu, and J. Du, "DeepIoT: Compressing deep neural network structures for sensing systems with a compressor-critic framework," in Proc. SenSys, 2017, pp. 1–4.
- J. Mao, X. Chen, K. W. Nixon, C. Krieger, and Y. Chen, "MoDNN: Local distributed mobile computing system for deep neural network," in Proc. Design, Autom. Test Eur. Conf. Exhibit. (DATE), Mar. 2017, pp. 1396–1401.
- Z. Zhao, K. M. Barijough, and A. Gerstlauer, "Deepthings: Distributed adaptive deep learning inference on resource-constrained iot edge clusters," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 37, no. 11, pp. 2348–2359, Nov. 2018.
- W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," IEEE Internet of Things Journal (IoT-J), vol. 3, no. 5, pp. 637–646, 2016.
- R. Stahl, Z. Zhao, D. Mueller-Gritschneider, A. Gerstlauer, and U. Schlichtmann, "Fully distributed deep learning inference on resource-constrained edge devices," in International Conference on Embedded Computer Systems . Springer, 2019, pp. 77–90.



THANKS